

Documentacion Taller 2

Integrantes:

Baruc Jeronimo triana bastidas 202416093

Juan Avendaño 202411658

Sara Hernandez Orjuela 202412681

Punto 1.

Esta había sido mi primera implementación. Aunque funcionaba, tenía una duda sobre la forma en que estaba realizando el desempate entre vecinos.

```

1 def hill_climbing(
2     problem: SmartGridOptimizationProblem,
3     initial_configuration: Configuration,
4     max_iterations: int = 500,
5 ) -> OptimizationResult:
6     """
7     Ejecuta ascenso de colina con mejora estricta.
8
9     Debe examinar todos los vecinos, seleccionar el de mayor puntaje y
10    conservar el orden entregado por el problema para desempatar. La búsqueda
11    termina cuando no existe una mejora estricta o se alcanza el límite.
12
13    Tips:
14    - problem.neighbors(current) retorna vecinos válidos en el orden que debe
15    usarse para desempatar.
16    - Cada llamada a configuration_score(...) cuenta como una evaluación.
17    - Inicialice los historiales con la configuración inicial y agregue solo las
18    mejoras aceptadas antes de retornar el OptimizationResult.
19    """
20    current_configuration = tuple(initial_configuration)
21    current_score = configuration_score(problem, current_configuration)
22
23    evaluations = 1
24    iterations = 0
25
26    history: list[Configuration] = [current_configuration]
27    score_history: list[float] = [current_score]
28
29    while iterations < max_iterations:
30        neighboring_configurations = problem.neighbors(current_configuration)
31
32        if not neighboring_configurations:
33            break
34
35        best_neighbor = neighboring_configurations[0]
36        best_neighbor_score = configuration_score(problem, best_neighbor)
37        evaluations += 1
38
39        for neighbor_configuration in neighboring_configurations[1:]:
40            neighbor_score = configuration_score(problem, neighbor_configuration)
41            evaluations += 1
42
43            if neighbor_score >= best_neighbor_score:
44                best_neighbor = neighbor_configuration
45                best_neighbor_score = neighbor_score
46
47        if best_neighbor_score <= current_score:
48            break
49
50        current_configuration = best_neighbor
51        current_score = best_neighbor_score
52
53        iterations += 1
54        history.append(current_configuration)
55        score_history.append(current_score)
56
57    return OptimizationResult(
58        best_configuration=current_configuration,
59        best_score=current_score,
60        evaluations=evaluations,
61        iterations=iterations,
62        history=history,
63        score_history=score_history,
64    )

```

Por lo que le mande este pront a Chat gpt:

Revisé mi implementación de hill_climbing y en general funciona, pero tengo una duda con el desempate entre vecinos. Estoy usando `>=` cuando comparo el puntaje del vecino con el mejor encontrado. ¿Eso respeta el requisito de conservar el orden de vecinos entregado por `problem.neighbors()` o debería cambiar algo?

Y la respuesta de chat fue esta:

No. Con `>=`, cuando aparecen dos vecinos con el mismo puntaje, el segundo reemplaza al primero, por lo que se pierde el orden de desempate entregado por `problem.neighbors()`.

Como el enunciado pide conservar ese orden, la comparación debe usar únicamente `>`. Así, un vecino solo reemplaza al mejor actual cuando su puntaje es estrictamente mayor y, en caso de empate, se conserva el primero encontrado.

El cambio necesario es:

```
if neighbor_score >= best_neighbor_score:
```

por:

```
if neighbor_score > best_neighbor_score:
```

Y a partir de esto cambie la implementación a esto:

```
def hill_climbing(
) -> OptimizationResult:
    """
    Ejecuta ascenso de colina con mejora estricta.

    Debe examinar todos los vecinos, seleccionar el de mayor puntaje y
    conservar el orden entregado por el problema para desempatar. La búsqueda
    termina cuando no existe una mejora estricta o se alcanza el límite.

    Tips:
    - problem.neighbors(current) retorna vecinos válidos en el orden que debe
      usarse para desempatar.
    - Cada llamada a configuration_score(...) cuenta como una evaluación.
    - Inicialice los historiales con la configuración inicial y agregue solo las
      mejoras aceptadas antes de retornar el OptimizationResult.
    """
    current_configuration = tuple(initial_configuration)
    current_score = configuration_score(problem, current_configuration)

    evaluations = 1
    iterations = 0

    history: list[Configuration] = [current_configuration]
    score_history: list[float] = [current_score]

    while iterations < max_iterations:
        neighboring_configurations = problem.neighbors(current_configuration)

        if not neighboring_configurations:
            break

        best_neighbor = neighboring_configurations[0]
        best_neighbor_score = configuration_score(problem, best_neighbor)
        evaluations += 1

        for neighbor_configuration in neighboring_configurations[1:]:
            neighbor_score = configuration_score(problem, neighbor_configuration)
            evaluations += 1

            if neighbor_score > best_neighbor_score:
                best_neighbor = neighbor_configuration
                best_neighbor_score = neighbor_score

        if best_neighbor_score <= current_score:
            break

        current_configuration = best_neighbor
        current_score = best_neighbor_score

        iterations += 1

        history.append(current_configuration)
        score_history.append(current_score)

    return OptimizationResult(
        best_configuration=current_configuration,
        best_score=current_score,
        evaluations=evaluations,
        iterations=iterations,
        history=history,
        score_history=score_history,
    )
```

a. Formulacion del problema

Componente	Formulación
Representación	Tupla binaria $(x_1, \dots, x_n)$, donde 1 indica que se instala un módulo y 0 que no.

Restricción de validez	La configuración debe contener exactamente k bits activos.
Función objetivo	Maximizar cobertura - redundancia - exposición.
Relación de vecindad	Retirar un módulo de un sitio seleccionado y ubicarlo en uno no seleccionado, manteniendo kmódulos.

La relación de vecindad mantiene siempre la validez de la configuración, ya que cada movimiento intercambia un sitio activo por uno inactivo sin modificar la cantidad total de módulos.

b. Funcion objetivo

```
def configuration_score(
    problem: SmartGridOptimizationProblem, configuration: Configuration
) -> float:
    """
    Combina cobertura, redundancia y exposición en un puntaje a maximizar.

    Tips:
    - Use problem.score_components(configuration); ya retorna cobertura,
      redundancia y exposición en ese orden.
    """
    coverage, redundancy, exposure = problem.score_components(configuration)
    return coverage - redundancy - exposure
```

La función `configuration_score` obtiene la cobertura, redundancia y exposición mediante `problem.score_components(configuration)`. El puntaje final se calcula como `coverage - redundancy - exposure`, por lo que se busca aumentar la cobertura y reducir las penalizaciones por redundancia y exposición.

c. Hill climbing

```

def hill_climbing(
    problem: SmartGridOptimizationProblem,
    initial_configuration: Configuration,
    max_iterations: int = 500,
) -> OptimizationResult:
    """
    Ejecuta ascenso de colina con mejora estricta.

    Debe examinar todos los vecinos, seleccionar el de mayor puntaje y
    conservar el orden entregado por el problema para desempatar. La búsqueda
    termina cuando no existe una mejora estricta o se alcanza el límite.

    Tips:
    - problem.neighbors(current) retorna vecinos válidos en el orden que debe
      usarse para desempatar.
    - Cada llamada a configuration_score(...) cuenta como una evaluación.
    - Inicialice los historiales con la configuración inicial y agregue solo las
      mejoras aceptadas antes de retornar el OptimizationResult.
    """
    current_configuration = tuple(initial_configuration)
    current_score = configuration_score(problem, current_configuration)

    evaluations = 1
    iterations = 0

    history: List[Configuration] = [current_configuration]
    score_history: List[float] = [current_score]

    while iterations < max_iterations:
        neighboring_configurations = problem.neighbors(current_configuration)

        if not neighboring_configurations:
            break

        best_neighbor = neighboring_configurations[0]
        best_neighbor_score = configuration_score(problem, best_neighbor)
        evaluations += 1

        for neighbor_configuration in neighboring_configurations[1:]:
            neighbor_score = configuration_score(problem, neighbor_configuration)
            evaluations += 1

            if neighbor_score > best_neighbor_score:
                best_neighbor = neighbor_configuration
                best_neighbor_score = neighbor_score

        if best_neighbor_score <= current_score:
            break

        current_configuration = best_neighbor
        current_score = best_neighbor_score

        iterations += 1

        history.append(current_configuration)
        score_history.append(current_score)

    return OptimizationResult(
        best_configuration=current_configuration,
        best_score=current_score,
        evaluations=evaluations,
        iterations=iterations,
        history=history,
        score_history=score_history,
    )

```

El algoritmo comienza desde una configuración válida y evalúa todos sus vecinos. En cada iteración selecciona el vecino de mayor puntaje y únicamente realiza el movimiento si representa una mejora estricta. La comparación utiliza > para conservar el primer vecino entregado por problem.neighbors() cuando existe un empate. La búsqueda termina cuando no existe una mejora estricta o se alcanza el límite de iteraciones.

evaluations cuenta cada llamada a configuration_score, mientras que iterations aumenta únicamente cuando se acepta una nueva configuración. Los historiales contienen la configuración inicial y las mejoras aceptadas.

d. Pruebas con diferentes configuraciones iniciales

Layout	Seed	Puntaje final	Evaluaciones	Iteraciones
local_peak	1	96.00	46	2
local_peak	15	96.00	31	1
plateau_grid	7	92.00	31	1
plateau_grid	1	94.00	46	2

```

➤ → RedCentinela git:(main) x python3 main.py -m optimization -a hillClimbing -l local_peak --seed 1 -t
hillClimbing | mejor=96.00 | promedio=96.00 | desviación=0.00 | evaluaciones_promedio=46.0 | tiempo_promedio=0.000234s

=== CONFIGURACIÓN FINAL ===
#####
%O..C..N..%
%...%...%
%N..C..O..%
%.....%
%..N...C.N%
%...%...%
%C..O..N..%
#####
Puntaje=96.00 | Evaluaciones=46 | Iteraciones=2
➤ → RedCentinela git:(main) x python3 main.py -m optimization -a hillClimbing -l local_peak --seed 15 -t
hillClimbing | mejor=96.00 | promedio=96.00 | desviación=0.00 | evaluaciones_promedio=31.0 | tiempo_promedio=0.000209s

=== CONFIGURACIÓN FINAL ===
#####
%O..C..N..%
%...%...%
%N..C..O..%
%.....%
%..N...C.N%
%...%...%
%C..O..N..%
#####
Puntaje=96.00 | Evaluaciones=31 | Iteraciones=1
➤ → RedCentinela git:(main) x python3 main.py -m optimization -a hillClimbing -l plateau_grid --seed 7 -t
hillClimbing | mejor=92.00 | promedio=92.00 | desviación=0.00 | evaluaciones_promedio=31.0 | tiempo_promedio=0.000233s

=== CONFIGURACIÓN FINAL ===
#####
%N.C.O.C%
%.....%
%N....N%
%..C.C..%
%N....O%
%.....%
%C.O.C.N%
#####
Puntaje=92.00 | Evaluaciones=31 | Iteraciones=1
➤ → RedCentinela git:(main) x python3 main.py -m optimization -a hillClimbing -l plateau_grid --seed 1 -t
hillClimbing | mejor=94.00 | promedio=94.00 | desviación=0.00 | evaluaciones_promedio=46.0 | tiempo_promedio=0.000315s

=== CONFIGURACIÓN FINAL ===
#####
%O.C.N.C%
%.....%
%N....O%
%..C.C..%
%N....N%
%.....%
%C.O.C.N%
#####
Puntaje=94.00 | Evaluaciones=46 | Iteraciones=2
○ → RedCentinela git:(main) x 

```

Efecto de la inicialización: En plateau_grid, la semilla 7 terminó con puntaje 92, mientras que la semilla 1 alcanzó 94. Por lo tanto, en la ejecución con semilla 7 el algoritmo se detuvo en una solución inferior al mejor valor observado en las pruebas. Esto evidencia que Hill Climbing depende de la configuración inicial y puede quedar detenido en una región subóptima.

Costo de búsqueda: En `local_peak`, ambas semillas alcanzaron el mismo puntaje de 96, pero con diferente número de iteraciones y evaluaciones. La semilla 15 requirió una sola mejora, mientras que la semilla 1 necesitó dos.

Máximos locales y mesetas: El algoritmo examina todos los vecinos, pero únicamente cambia de configuración cuando encuentra una mejora estricta. Por esta razón puede detenerse aunque exista una configuración mejor en otra región del espacio de búsqueda.

2

Código Inicial:

```
def one_point_crossover(  
    if len(parent1) != len(parent2):  
        raise ValueError("Los padres deben tener la misma longitud")  
    if len(parent1) < 2:  
        return parent1, parent2  
  
    corte = rng.randint(0, len(parent1))  
    hijo1 = parent1[:corte] + parent2[corte:]  
    hijo2 = parent2[:corte] + parent1[corte:]  
    return hijo1, hijo2
```

```

def simulated_annealing(
    rng = rng or random.Random()
    minimum_temperature = 1e-9
    actual = initial_configuration
    puntaje_actual = configuration_score(problem, actual)

    historial = [actual]
    historial_puntajes = [puntaje_actual]
    evaluaciones = 1
    iteraciones = 0
    for i in range(1, max_iterations + 1):
        temperatura = cooling_schedule(initial_temperature, cooling_rate, i)
        if temperatura < minimum_temperature:
            break

        vecinos = problem.neighbors(actual)
        candidato = random.choice(vecinos)
        puntaje_candidato = configuration_score(problem, candidato)
        evaluaciones = evaluaciones + 1
        iteraciones = iteraciones + 1

        delta = puntaje_candidato - puntaje_actual
        probabilidad = math.exp(delta / temperatura)

        if random.random() < probabilidad:
            actual = candidato
            puntaje_actual = puntaje_candidato
            historial.append(actual)
            historial_puntajes.append(puntaje_actual)

    return OptimizationResult( best_configuration=actual, best_score=puntaje_actual,
                               evaluations=evaluaciones, iterations=iteraciones, history=historial,
                               score_history=historial_puntajes,
    )

```

Prompt:

Revisa y corrige este código (código adjunto). Explicame por qué simulated_annealing da resultados distintos en cada corrida y el puntaje final suele ser peor que el inicial.

Respuesta:

-random.choice y random.random usan el generador global del módulo, no el rng recibido: de ahí la no reproducibilidad. Además optimization/runner.py envuelve el rng en un RecordingRandom para reconstruir la animación, así que usar el generador global también rompe la visualización.

- $T_0 * \alpha * \text{iteration}$ es un enfriamiento **lineal creciente**, no geométrico: la temperatura sube en vez de bajar. Debe ser $T_0 * \alpha^{**\text{iteration}}$.

-El retorno es el estado actual, no el mejor encontrado. Hay que llevarlos por separado.

-Al corregir el enfriamiento, $\text{math.exp}(\text{delta} / T)$ con $\text{delta} > 0$ y T diminuta **desborda**.

Código Final:

```
centinela > algorithms > optimization.py > simulated_annealing
def cooling_schedule(initial_temperature: float, cooling_rate: float, iteration: int) -> float:
    return initial_temperature * (cooling_rate ** iteration)
```

```
def simulated_annealing(
    rng = rng or random.Random()
    minimum_temperature = 1e-9
    current = tuple(initial_configuration)
    current_score = configuration_score(problem, current)
    evaluations = 1
    iterations = 0
    best_configuration = current
    best_score = current_score
    history: list[Configuration] = [current]
    score_history: list[float] = [current_score]
    for step in range(max_iterations):
        temperature = cooling_schedule(initial_temperature, cooling_rate, step)
        if temperature <= minimum_temperature:
            break
        neighbors = problem.neighbors(current)
        if not neighbors:
            break
        candidate = rng.choice(neighbors)
        candidate_score = configuration_score(problem, candidate)
        evaluations += 1
        iterations += 1
        delta = candidate_score - current_score
        if delta > 0:
            accepted = True
        else:
            accepted = rng.random() < math.exp(delta / temperature)

        if accepted:
            current = candidate
            current_score = candidate_score
            if current_score > best_score:
                best_configuration = current
                best_score = current_score
```

```
def simulated_annealing(
    history.append(current)
    score_history.append(current_score)

    return OptimizationResult(
        best_configuration=best_configuration,
        best_score=best_score,
        evaluations=evaluations,
        iterations=iterations,
        history=history,
        score_history=score_history,
    )
```

Evidencia: efecto de T0 y alpha

risk_corridor

T0	Alpha	mejor	promedio	desviación	Evaluaciones_prom	Tiempo_prom (s)
20	0,97	116	114,13	2,55	501	0,011929
20	0,8	116	112,2	3,67	108	0,002475
2	0,97	116	111,6	3,48	501	0,012725
60	0,995	116	115,93	0,25	501	0,012123

local_peak

T0	Alpha	mejor	promedio	desviación	Evaluaciones_prom	Tiempo_prom (s)
20	0,97	96	94,27	4,42	501	0.007409
20	0,8	96	95	3,74	108	0.001252
2	0,97	96	95	3,74	501	0.005594
60	0,995	96	96	0	501	0.005911

Enfriamiento lento (alpha = 0.995) domina: Mantiene temperatura útil durante las 500 iteraciones, acepta deterioros el tiempo suficiente para cruzar las cuencas y termina prácticamente siempre en el óptimo, con desviación ≈ 0 . Es la configuración que más claramente supera al ascenso de colina (115.93 vs. 111.67 de promedio en risk_corridor).

Enfriamiento rápido (alpha = 0.80): La temperatura cae bajo $1e-9$ hacia la iteración ~ 107 , así que el algoritmo se detiene por temperatura mínima con ~ 108 evaluaciones. Es cinco

veces más barato, pero degenera en un ascenso de colina estocástico: en `risk_corridor` baja a 112.20 de promedio con la mayor desviación (3.67).

Temperatura inicial baja ($T_0 = 2$): Con puntajes cuyos delta típicos valen varias unidades, $\exp(\text{delta}/T)$ es casi 0 desde el inicio, así que casi no acepta deterioros. Es el peor promedio en `risk_corridor` (111.60) y gasta igual 501 evaluaciones. Confirma que T_0 debe estar en la escala de los delta del problema, no ser un número pequeño arbitrario.

La aleatoriedad importa: Con 15 corridas por configuración, las diferencias de promedio entre (60, 0.995) y el resto son consistentes y no producto de una corrida particular (la desviación de la mejor configuración es 0.25 frente a 2.5–3.7 de las demás).

3.

Código Inicial:

```
def one_point_crossover(
    parent1: Configuration, parent2: Configuration, rng: random.Random
) -> tuple[Configuration, Configuration]:
    if len(parent1) != len(parent2):
        raise ValueError("Los padres deben tener la misma longitud")
    if len(parent1) < 2:
        return parent1, parent2

    corte = rng.randint(0, len(parent1))
    hijo1 = parent1[:corte] + parent2[corte:]
    hijo2 = parent2[:corte] + parent1[corte:]
    return hijo1, hijo2

def swap_mutation(
    individual: Configuration, mutation_probability: float, rng: random.Random
) -> Configuration:
    individuo = list(individual)
    if rng.random() < mutation_probability:
        indice = rng.randint(0, len(individuo) - 1)
        if individuo[indice] == 1:
            individuo[indice] = 0
        else:
            individuo[indice] = 1
    return individuo
```

```

def genetic_algorithm(
    problem: SmartGridOptimizationProblem,
    population_size: int = 40,
    generations: int = 100,
    mutation_probability: float = 0.05,
    elite_size: int = 2,
    rng: random.Random | None = None,
) -> OptimizationResult:
    rng = random.Random()
    if population_size < 2:
        raise ValueError("La poblacion debe tener al menos dos individuos")
    if generations < 0:
        raise ValueError("El numero de generaciones no puede ser negativo")
    if not 0.0 <= mutation_probability <= 1.0:
        raise ValueError("La probabilidad de mutacion debe estar entre 0 y 1")
    if not 0 <= elite_size <= population_size:
        raise ValueError("elite_size debe estar entre 0 y population_size")

    poblacion = problem.initial_population(population_size, rng)
    puntajes = [configuration_score(problem, x) for x in poblacion]
    evaluaciones = population_size

    historial = []
    historial_puntajes = []

    for generacion in range(generations):
        nueva_poblacion = []

```

```

def genetic_algorithm(
    while len(nueva_poblacion) < population_size:
        padre1 = problem.tournament_select(poblacion, puntajes, rng)
        padre2 = problem.tournament_select(poblacion, puntajes, rng)
        hijo1, hijo2 = one_point_crossover(padre1, padre2, rng)

        hijo1 = swap_mutation(hijo1, mutation_probability, rng)
        hijo2 = swap_mutation(hijo2, mutation_probability, rng)

        hijo1 = problem.repair_configuration(hijo1, rng)
        hijo2 = problem.repair_configuration(hijo2, rng)

        nueva_poblacion.append(hijo1)
        nueva_poblacion.append(hijo2)

    poblacion = nueva_poblacion
    puntajes = [configuration_score(problem, x) for x in poblacion]
    evaluaciones = evaluaciones + len(poblacion)

    indice_mejor = puntajes.index(max(puntajes))
    historial.append(poblacion[indice_mejor])
    historial_puntajes.append(puntajes[indice_mejor])

    indice_mejor = puntajes.index(max(puntajes))

    return OptimizationResult(
        best_configuration=poblacion[indice_mejor],
        best_score=puntajes[indice_mejor],
        evaluations=evaluaciones,
        iterations=generations,
        history=historial,
        score_history=historial_puntajes,
    )

```

Prompt:

Revisa y corrige este código (código adjunto). Explicame por qué el algoritmo genético converge a un puntaje y después lo pierde, y tampoco respeta la semilla.

Respuesta:

-rng = random.Random() **descarta** el generador recibido. Por eso la semilla no sirve.

-rng.randint(0, len(parent1)) admite los cortes 0 y n, que no mezclan nada: devuelven a los padres intercambiados.

-swap_mutation invierte un bit, lo que cambia el número de módulos y viola la restricción de validez. Un *swap* debe apagar un bit activo y encender uno inactivo, así la cardinalidad se conserva. Además, devuelve una lista, no una tupla.

-El orden está invertido: se muta y después se repara, así que la reparación deshace la mutación de forma aleatoria. Debe ser cruce → reparación → mutación.

-No hay elitismo (elite_size se ignora) y se retorna el mejor de la última generación, no el mejor de toda la ejecución.

Código Final:

```
def one_point_crossover(  
    """  
    if len(parent1) != len(parent2):  
        raise ValueError("Los padres deben tener la misma longitud")  
    if len(parent1) < 2:  
        return parent1, parent2  
  
    cut = rng.randint(1, len(parent1) - 1)  
    child1 = tuple(parent1[:cut]) + tuple(parent2[cut:])  
    child2 = tuple(parent2[:cut]) + tuple(parent1[cut:])  
    return child1, child2
```

```
def swap_mutation(  
    if rng.random() >= mutation_probability:  
        return tuple(individual)  
  
    active = [index for index, bit in enumerate(individual) if bit]  
    inactive = [index for index, bit in enumerate(individual) if not bit]  
    if not active or not inactive:  
        return tuple(individual)  
  
    remove = rng.choice(active)  
    add = rng.choice(inactive)  
    mutated = list(individual)  
    mutated[remove] = 0  
    mutated[add] = 1  
    return tuple(mutated)
```

```

def genetic_algorithm(
    rng = rng or random.Random()
    if population_size < 2:
        raise ValueError("La población debe tener al menos dos individuos")
    if generations < 0:
        raise ValueError("El número de generaciones no puede ser negativo")
    if not 0.0 <= mutation_probability <= 1.0:
        raise ValueError("La probabilidad de mutación debe estar entre 0 y 1")
    if not 0 <= elite_size <= population_size:
        raise ValueError("elite_size debe estar entre 0 y population_size")

    population = problem.initial_population(population_size, rng)
    scores = [configuration_score(problem, individual) for individual in population]
    evaluations = len(population)

    best_index = max(range(len(population)), key=lambda index: scores[index])
    best_configuration = population[best_index]
    best_score = scores[best_index]

    history: list[Configuration] = [best_configuration]
    score_history: list[float] = [best_score]

    offspring_needed = population_size - elite_size

    for _ in range(generations):
        # Elitismo: los mejores individuos pasan intactos a la nueva población.
        ranking = sorted(range(len(population)), key=lambda index: scores[index], reverse=True)
        elite_indices = ranking[:elite_size]
        new_population = [population[index] for index in elite_indices]
        new_scores = [scores[index] for index in elite_indices]

```

```

def genetic_algorithm(
    while len(offspring) < offspring_needed:
        parent1 = problem.tournament_select(population, scores, rng)
        parent2 = problem.tournament_select(population, scores, rng)
        child1, child2 = one_point_crossover(parent1, parent2, rng)
        for child in (child1, child2):
            if len(offspring) >= offspring_needed:
                break
            repaired = problem.repair_configuration(child, rng)
            mutated = swap_mutation(repaired, mutation_probability, rng)
            offspring.append(mutated)

        for child in offspring:
            new_population.append(child)
            new_scores.append(configuration_score(problem, child))
            evaluations += 1

        population = new_population
        scores = new_scores

        generation_best = max(range(len(population)), key=lambda index: scores[index])
        if scores[generation_best] > best_score:
            best_configuration = population[generation_best]
            best_score = scores[generation_best]

        history.append(best_configuration)
        score_history.append(best_score)

    return OptimizationResult(
        best_configuration=best_configuration,
        best_score=best_score,
        evaluations=evaluations,
        iterations=generations,
        history=history,
        score_history=score_history
    )

```

Evidencia: efecto de T0 y Alpha

10 corridas por configuración.

risk_corridor

Población	Generaciones	Mutación	Mejor	Promedio	Desviación	Evaluaciones_Prom	Tiempo_Prom (s)
40	100	0,05	283	282	0,98	3840	0,094821
40	100	0,3	283	282	0,92	3840	0,093660
10	100	0,05	283	278	2,25	810	0,019260
40	25	0,05	283	281	1,4	990	0,021489

balanced_city

Población	Generaciones	Mutación	Mejor	Promedio	Desviación	Evaluaciones_Prom	Tiempo_Prom (s)
40	100	0,05	283	281,9	1,14	3840	0,110565
40	100	0,3	283	282,1	1,14	3840	0.118612
10	100	0,05	283	279,2	2,68	810	0,019647
40	25	0,05	283	281,7	1,42	990	0,024526
40	100	0,05	283	282,6	0,8	4040	0,096113

Tamaño de la población es el parámetro más influyente. Bajar de 40 a 10 individuos reduce el costo casi $5\times$ (810 vs. 3840 evaluaciones) pero pierde diversidad: en `balanced_city` es la única configuración que no alcanza el óptimo en ninguna corrida (mejor 282.00) y su desviación se duplica (2.68 vs. 1.14). Con poblaciones pequeñas el torneo converge prematuramente a un solo linaje.

Probabilidad de mutación: subirla de 0.05 a 0.30 no cuesta nada en evaluaciones y mejora o iguala el promedio (`risk_corridor`: 115.40 $\rightarrow$ 116.00 en las 10 corridas, desviación 0). Con `swap_mutation` la mutación es el único operador que reintroduce diversidad después de que el cruce + reparación homogeneizan la población, así que una tasa baja deja al GA estancado en la cuenca a la que llegó.

Generaciones: pasar de 100 a 25 no cambia la calidad (115.40 y 281.70) con 1/4 del costo. La curva de `score_history` es plana mucho antes de la generación 25. Gastar 100 generaciones es desperdicio en estos mapas.

Comparado con los otros dos algoritmos, el GA es el más caro (≈ 3840 evaluaciones frente a ~ 40 –250 del ascenso de colina y 501 del recocido) y el más consistente: nunca terminó por debajo de 279 en ningún layout ni semilla probados.

Punto 4

a) Formulación del juego

Componente	En el proyecto
Estado inicial	El GameState que se carga del layout: posiciones del defensor y del intruso, terminales pendientes y puntaje.
Jugador en turno	Se alternan: defensor (índice 0, MAX) y luego intruso (índice 1, MIN). El siguiente es $(\text{agente} + 1) \% \text{state.get_num_agents}()$.
Acciones	<code>state.get_legal_actions(agente)</code> : los movimientos legales del agente en turno, en el orden que entrega el estado.

Resultado (sucesor)	state.generate_successor(agente, accion): el estado que queda después de aplicar la acción.
Prueba terminal	state.is_win(). El defensor activó las terminales o state.is_lose(). El intruso lo interceptó.
Utilidad	+1000 si gana y -1000 si pierde (vía evaluation_function). En los cortes por profundidad, el valor que da evaluation_function.

b) Implementación

El código tiene dos partes.

get_action (la raíz):

1. Reinicia el contador y cuenta la raíz.
2. Recorre las acciones del defensor en orden.
3. A cada sucesor le pide su valor con self.depth - 1, porque usar una acción gasta un ply.
4. Se queda con la acción de mayor valor.

minimax_value(state, agente, profundidad_restante) (la recursión):

1. Cuenta el estado una vez.
2. Si es terminal, si la profundidad llegó a 0 o si el agente no tiene movimientos, retorna evaluation_function(state).
3. Si no, expande los hijos y toma el máximo cuando juega el agente 0 o el mínimo cuando juega el 1.

Con esta regla, depth=1 solo mira una acción de MAX y depth=2 agrega la respuesta de MIN.

c) MAX, MIN y empates

El agente 0 se modela como MAX: empieza en $-\infty$ y se queda con el valor más alto. El agente 1 se modela como MIN: empieza en $+\infty$ y se queda con el valor más bajo.

Para los empates, la comparación en la raíz es valor $>$ mejor_valor (estricta). Una acción posterior con el mismo valor no reemplaza a la anterior, así que se conserva la primera según el orden de get_legal_actions.

d) Comparación de profundidades

Layout	Profundidad	Acción inicial	Resultado	Nodos 1. ^a jugada	Nodos totales de la partida
single_terminal	2	South	Límite de rondas (60), puntaje -60	13	991
single_terminal	4	South	Victoria en 9 rondas, puntaje 1000	121	1357
tiny_defense	2	North	Límite de rondas (60), puntaje -60	13	1107
tiny_defense	4	North	Límite de rondas (60), puntaje -60	143	15384
bottleneck	2	West	Límite de rondas (60), puntaje -60	9	983
bottleneck	4	West	Límite de rondas (60), puntaje -60	89	10627
deceptive_distance	2	North	Límite de rondas (60), puntaje -60	17	1140
deceptive_distance	4	North	Límite de rondas (60), puntaje -60	213	16168

Punto 5:

a) AlphaBetaAgent

Tiene la misma estructura que Minimax, con un método auxiliar alphabeta_value que además recibe alpha y beta:

- **Nodo MAX (defensor):** si valor $\geq$ beta retorna de inmediato, porque MIN ya tiene algo igual o mejor y nunca dejará llegar ahí. Si no, actualiza alpha.
- **Nodo MIN (intruso):** si valor $\leq$ alpha retorna, porque MAX ya tiene algo igual o mejor asegurado. Si no, actualiza beta.

- **Raíz:** empieza con $\alpha = -\infty$ y $\beta = +\infty$, y va subiendo alpha con la mejor acción encontrada.
- **Conteo:** nodes_evaluated se reinicia, cuenta la raíz y suma 1 por cada estado procesado, igual que en el punto 4.

b) Función de evaluación

Para estados terminales retorna la evaluación base (+1000 o -1000). Para el resto calcula:

$$\text{valor} = \text{puntaje} - 60 \cdot (\text{terminales pendientes}) - 4 \cdot (\text{distancia a la terminal objetivo}) + 2 \cdot (\text{distancia al intruso, con tope 6}) - 100 \text{ si el intruso está a } \leq 1 \text{ paso} + 1 \cdot (\text{movilidad})$$

Al final lo recorta a $[-999, 999]$. Todas las distancias son reales por el mapa (layout.distance), y los caminos inexistentes (infinito) se reemplazan por un número grande o por el tope.

c) Verificación y comparación de nodos

Nodos en los mapas pedidos:

Mapa	Prof.	Acción (MM / AB)	Resultado	Nodos 1. ^a jugada MM → AB	Nodos de la partida MM → AB
dual_terminal	2	Stop / Stop	Victoria, 22 rondas	17 → 17 (0 %)	432 → 311 (-28,0 %)
dual_terminal	4	East / East	Victoria, 5 rondas	245 → 91 (-62,9 %)	1024 → 289 (-71,8 %)
medium_defense	2	West / West	Victoria, 30 rondas	25 → 22 (-12,0 %)	608 → 438 (-28,0 %)
medium_defense	4	West / West	Victoria, 53 rondas	457 → 188 (-58,9 %)	12082 → 4880 (-59,6 %)

En los cuatro casos las dos partidas son idénticas jugada por jugada.

No se llega al caso ideal $O(b^{(m/2)})$ porque el orden de las acciones es fijo (North, South, East, West, Stop), no de mejor a peor.

d) Justificación de características y pesos

Característica	Peso	Por qué
Puntaje acumulado	1	Ya premia activar terminales (+100) y castiga cada movimiento (-1), así que desempata a favor de rutas cortas.

Terminales pendientes	-60 c/u	Refuerza que activar una terminal siempre compense. Al activarla, la distancia al siguiente objetivo puede saltar, y $100 + 60$ supera ese salto multiplicado por 4.
Distancia a la terminal objetivo	-4 por paso	Da la "pendiente" que le faltaba a la evaluación base. Sin ella todos los cortes valían lo mismo y el defensor oscilaba.
Distancia al intruso (tope 6)	+2 por paso	Premia mantener separación. Más allá de 6 pasos el intruso deja de ser una amenaza cercana y no conviene seguir huyendo.
Captura inmediata (≤ 1 paso)	-100	Marca el riesgo real de ser interceptado en la siguiente jugada, aunque la búsqueda no alcance a ver el -1000.
Movilidad	+1 por acción	Desempata a favor de casillas con salidas y en contra de callejones sin salida.

5. Reflexión sobre el trabajo realizado

6a. Analisis de algoritmos y resultados

a. ¿Qué características de los layouts explican las diferencias entre ascenso de colina, recocido simulado y algoritmo genético?

Los resultados muestran que la topología del layout modifica de forma importante el comportamiento de los algoritmos. En mapas como `plateau_grid`, el ascenso de colina puede detenerse porque exige una mejora estricta y no puede atravesar una meseta. Esto se observó al obtener puntajes finales de 92 y 94 a partir de distintas configuraciones iniciales. En `risk_corridor`, la combinación entre cobertura y exposición genera varias regiones de distinta calidad. El recocido simulado tiene ventaja porque puede aceptar temporalmente movimientos que disminuyen el puntaje y así salir de una cuenca local. Con un enfriamiento lento alcanzó un promedio de 115.93 frente a 111.67 del ascenso de colina. El algoritmo genético explora varias regiones simultáneamente mediante una población. Esto resulta útil en layouts con varias cuencas, como `risk_corridor` y `balanced_city`, aunque requiere muchas más evaluaciones.

b. ¿Qué efecto tuvieron la inicialización, aleatoriedad y parámetros?

En ascenso de colina, la configuración inicial determina la región del espacio que se explora. Como el algoritmo no acepta movimientos laterales ni deterioros, distintas semillas pueden conducir a soluciones finales diferentes. En recocido simulado, la temperatura inicial y el factor de enfriamiento controlan cuánto tiempo se mantiene la exploración. Un enfriamiento lento ($\alpha = 0.995$) produjo los resultados más estables, mientras que un enfriamiento rápido ($\alpha = 0.80$) redujo el costo

pero también la calidad promedio. Una temperatura inicial demasiado baja hizo que casi no se aceptaran deterioros.

En el algoritmo genético, el tamaño de la población, la mutación y el número de generaciones controlan la diversidad y el costo. Una población pequeña redujo fuertemente las evaluaciones, pero aumentó la variabilidad y produjo peores resultados. La mutación ayudó a recuperar diversidad, mientras que el elitismo evitó perder las mejores soluciones ya encontradas.

c. ¿Qué compromiso existe entre calidad y costo computacional?

Algoritmo	Configuración representativa en risk_corridor	Promedio	Mejor	Evaluaciones aprox.
Ascenso de colina	15 inicializaciones	111.67	116	131.7
Recocido simulado	$T_0=60$, $\alpha=0.995$	115.93	116	501
Algoritmo genético	Población 40, 100 gen., mutación 0.30	116.00	116	3840

Existe un compromiso claro entre calidad y costo. Ascenso de colina fue el algoritmo más económico, pero también el más sensible a la inicialización y el que presentó el menor promedio. Recocido simulado aumentó el número de evaluaciones, pero obtuvo soluciones mucho más consistentes y cercanas al mejor valor observado. El algoritmo genético fue el más costoso, pero con una población suficiente y una tasa de mutación adecuada alcanzó el mejor valor de forma consistente. Por lo tanto, una mayor exploración mejora la robustez de la solución, pero incrementa considerablemente el costo computacional.

- d. identifique un caso en el que cambiar la profundidad modifique la decisión o el resultado del defensor. Explique por qué ocurre y relacione su respuesta con la función de evaluación y el efecto horizonte.

En single_terminal, aumentar la profundidad de 2 a 4 plies no modificó la acción inicial, que fue South, pero sí cambió completamente el resultado de la partida. Con profundidad 2, la ejecución alcanzó el límite de 60 rondas con puntaje -60; con profundidad 4, el defensor consiguió una victoria en 9 rondas con puntaje 1000.

La diferencia ocurre porque una mayor profundidad amplía el horizonte de búsqueda. Con pocos plies, Minimax debe decidir a partir de la función de evaluación en estados todavía lejanos al objetivo. Al aumentar la profundidad puede anticipar más movimientos del intruso y reconocer las consecuencias futuras de las acciones actuales. El costo es un crecimiento considerable del número de nodos explorados: en este caso, la primera decisión pasó de 13 a 121 nodos.

e. Relación entre Minimax, alfa-beta, profundidad y orden de acciones

Alfa-beta debe producir la misma decisión que Minimax porque no cambia los valores del árbol: únicamente evita explorar ramas que ya se sabe que no pueden modificar la elección final. Para que esto se cumpla, ambos algoritmos deben utilizar la misma profundidad, función de evaluación y orden de acciones.

Esto se verificó en `dual_terminal` y `medium_defense`, donde Minimax y alfa-beta escogieron las mismas acciones y las partidas fueron idénticas jugada por jugada. Sin embargo, alfa-beta procesó menos nodos. Por ejemplo, en `dual_terminal` con profundidad 4, la primera decisión pasó de 245 nodos con Minimax a 91 con alfa-beta, y la partida completa de 1024 a 289 nodos. En `medium_defense` con profundidad 4, el total disminuyó de 12082 a 4880 nodos.

La profundidad aumenta rápidamente el costo porque cada nivel agrega nuevas ramas al árbol. La eficacia de alfa-beta también depende del orden de las acciones: cuanto antes se exploran las mejores alternativas, antes se ajustan α y β y más ramas pueden podarse. En este proyecto el orden es fijo y no está organizado de mejor a peor, por lo que se obtiene una reducción importante, aunque no la poda ideal.

6b. Reflexión sobre el uso de inteligencia artificial

Durante el desarrollo utilizamos herramientas de inteligencia artificial principalmente como apoyo para revisar implementaciones que ya habíamos planteado y detectar errores lógicos difíciles de identificar únicamente observando la ejecución. En ascenso de colina, por ejemplo, la revisión permitió identificar que usar $\geq$ modificaba el criterio de desempate requerido. En recocido simulado ayudó a detectar problemas relacionados con el uso del generador aleatorio, el programa de enfriamiento y la diferencia entre el estado actual y el mejor estado encontrado. En el algoritmo genético permitió revisar aspectos como la conservación de la cantidad de módulos durante la mutación, el orden de los operadores y el elitismo.

Los cambios sugeridos no tuvieron todos la misma importancia. Algunos fueron correcciones pequeñas, como el desempate de Hill Climbing, mientras que otros fueron fundamentales para que el algoritmo cumpliera correctamente la especificación, especialmente en recocido simulado y algoritmo genético. La principal lección fue que una sugerencia de IA no debe aceptarse directamente: fue necesario contrastarla con el

enunciado, revisar las funciones entregadas por el proyecto y ejecutar pruebas para comprobar su efecto.

La parte más difícil al formular los prompts fue explicar exactamente qué comportamiento se estaba observando y qué condiciones del proyecto debían conservarse. Los prompts fueron más útiles cuando incluían el síntoma, el fragmento de código y una pregunta concreta sobre la causa del problema, en lugar de pedir simplemente una solución completa. En una sustentación podríamos explicar la solución final a partir de las reglas de cada algoritmo, los cambios realizados y las pruebas utilizadas para verificar su funcionamiento.